

CS3212 Algorithms

Aya Zirikly
10/08/2026

Reminders

- Midterm 10/15
- Extra office hours 10/14
 - 2:15-3pm SEH 2880

Hashing Functions

What Makes a Good Hash Function?

A hash function $h(k)$ maps a large universe of keys U into m slots in a hash table $T[0 \dots m-1]$.

- **Primary Objective:** Distribute keys uniformly across all m slots to minimize collisions.
- **Deterministic Requirement:** The same key k must *a/ways* hash to the exact same slot $h(k)$.
- **Fast Computation:** Computing $h(k)$ must take $O(1)$ time.

Reminder

Simple Uniform Hashing Assumption (SUHA)

Each key is equally likely to hash to any of the m slots, independently of where every other key has hashed.

- Under SUHA, with n keys and m slots, the load factor $\alpha = n / m$ represents the average number of keys per slot.
- Expected search/insert time with chaining: **$O(1 + \alpha)$** .

The Division Method

$$h(k) = k \bmod m$$

We map key k to the remainder of k divided by table size m .

How to Choose Table Size m

- **Rule:** Choose m to be a **prime number** not too close to an exact power of 2 or 10.

Pros & Cons

- **Pros:** Extremely fast (single CPU division instruction), intuitive.
- **Cons:** Poor performance if m is chosen poorly or if keys share common factors with m .

Why avoid powers of 2?

The Binary Math Behind $k \bmod 2^p$

In base-10 (our normal counting system):

- What is $123,456 \bmod 10^3$ (1000)?
- It is simply the **last 3 digits**: 456.
- It doesn't matter if the number starts with 123 or 999; the leading digits are completely thrown away!

In binary (base-2), the exact same rule applies:

- Taking $k \bmod 2^p$ extracts the **last p binary bits** of k .

Example

Suppose your hash table size is $m = 2^3 = 8$ slots ($p = 3$).

Now imagine your keys are memory addresses or auto-incrementing database IDs that happen to share a pattern in their lower bits (e.g., all numbers are multiples of 8, or all end in the binary pattern **011**):

Key (k in Decimal)	Key (k in Binary)	Hash Result: $h(k)=k \bmod 8$	Binary Result (Bottom 3 Bits)
19	00010 011	$19 \bmod 8 = 3$	11
51	00011 011	$51 \bmod 8 = 3$	11
83	00101 011	$83 \bmod 8 = 3$	11
115	00111 011	$115 \bmod 8 = 3$	11

Why Choosing a Prime Number m Fixes This

When you choose m to be a **prime number** (that is not close to a power of 2):

- Calculating $k \bmod m$ forces the modulo arithmetic to **depend on every single bit** of the key k , both high and low.

The Multiplication Method

$$h(k) = \text{floor}(m * ((k * A) \bmod 1))$$

Why Choice of m is flexible here?

- Unlike the Division Method, m does not need to be prime.
- We typically choose $m = 2^p$ (a power of 2).
- **Knuth's Optimal Choice:** Knuth suggests using the Golden Ratio for A :

$$A = (\text{sqrt}(5) - 1) / 2 = 0.6180339887\dots$$

The Multiplication Method

$$h(k) = \text{floor}(m * (k * A \bmod 1))$$

Why Choice of m is flexible here?

- Unlike the Division Method, m **does not need to be prime**.
- We typically choose $m = 2^p$ (a power of 2).
- **Knuth's Optimal Choice:** Knuth suggests using the Golden Ratio for A :

$$A = (\text{sqrt}(5) - 1) / 2 = 0.6180339887\dots$$

- ❑ Multiplier A acts like a bit-mixer.
- ❑ Multiplying by m simply stretches that fraction across the range $[0, m)$.
- ❑ Taking the floor truncates it to an integer index in $\{0, 1, \dots, m-1\}$.
- ❑ Because m can be anything, computer architects choose $m = 2^p$ (a power of 2) to unlock massive performance gains in hardware

Hashing Non-Integer Keys (Polynomial Rolling Hash)

Converting Strings to Integers

Hash functions require numeric inputs. To hash a string $S = s[0] s[1] \dots s[L-1]$, we treat it as a polynomial evaluation in base p .

$$h(S) = (s[0]*p^{(L-1)} + s[1]*p^{(L-2)} + \dots + s[L-1]*p^0) \bmod m$$

Where p is a small prime (typically $p = 31$ or $p = 37$ for ASCII text).

Universal Hashing (Defeating Adversarial Inputs)

The Problem with Static Hash Functions

If a fixed hash function h is known, an adversary can intentionally feed a set of keys that all hash to slot 0. This degrades operations from $O(1)$ down to **$O(n)$ worst-case time**.

The Solution: Universal Hashing

Instead of using one fixed function, choose a hash function **at random** from a carefully designed family of functions H at runtime.

Universal Hashing (Defeating Adversarial Inputs)

Definition of a Universal Family $\mathcal{H}$

A collection of hash functions $\mathcal{H}$ mapping universe U to $\{0, 1, \dots, m-1\}$ is **universal** if for every pair of distinct keys $k_1 \neq k_2$:

$$\text{Probability that } h(k_1) == h(k_2) \leq 1 / m$$

Constructing a Universal Family (CLRS Section 11.3.3)

Choose a prime $p > \max(k)$. Define $h_{(a,b)}(k)$ as:

$$h_{(a,b)}(k) = ((a * k + b) \bmod p) \bmod m$$

Where a is chosen randomly from $\{1, 2, \dots, p-1\}$ and b from $\{0, 1, \dots, p-1\}$.

When does the randomness happen?

The randomness **does NOT** mean that every time you call `hash("apple")`, it gives a different random bucket. If it did, you could never find your data again!

Instead:

1. **At program startup**, the system rolls a virtual die and picks **one** hash function h at random from a family of good functions.
2. **For the entire rest of the execution**, that chosen function h is **100% fixed and deterministic**.
3. `hash("apple")` will return slot 42 every single time during that run.

Applications

Databases & Compiler Compilers (Perfect Hashing for Static Sets)

- **The Problem:** Compilers (like GCC, Clang, or language parsers) need to check if an identifier is a reserved keyword (e.g., `if`, `else`, `while`, `return`). Database query engines similarly manage static dictionaries (e.g., country codes or fixed enum mappings).
- **Universal Hashing Solution:** They use **2-Level Perfect Hashing** (CLRS Section 11.5), which relies on Universal Hashing at both levels:
 - **Level 1:** Uses Universal Hashing to partition N static keys into N buckets.
 - **Level 2:** For any bucket that gets k_i collisions, it builds a sub-hash table of size k_i^2 using a secondary universal hash function.
 - **Result:** Guaranteed **zero collisions** and **absolute $O(1)$ worst-case search time** with $O(N)$ total space. Tools like `gperf` generate C/C++ code using this exact strategy.

Summary

Method	Formula / Strategy	Table Size m Requirement	Key Advantage
Division Method	$h(k) = k \bmod m$	Must be Prime (not near $2p$)	Extremely simple, single CPU instruction
Multiplication Method	$h(k) = \text{floor}(m * (k * A \bmod 1))$	Flexible (usually $m=2p$)	Fast binary bit-shift implementation
Polynomial String Hash	$\text{Sum}(s[i] * p^{(L-1-i)}) \bmod m$	Prime	Handles arbitrary length text / compound objects
Universal Hashing	Randomized h from family H	Prime $p > \text{max_key}$	Guarantees expected $O(1)$ time against adversaries

What is a Trie? (Beyond Key Lookup)

The Limitation of Hash Tables:

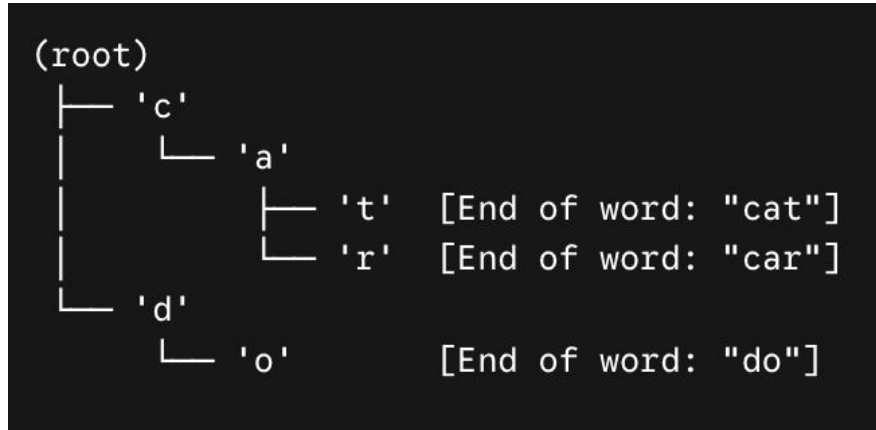
- Hash tables can find exact keys instantly (`Lookup( "apple" )`), but **cannot** answer prefix queries like:
"Find all words starting with 'app'".

Digital Search Tree / Trie (Prefix Tree):

- A tree structure where nodes represent individual characters/bits rather than full keys.
- Keys that share common prefixes share the same parent path in the tree.

How Tries Work: Prefix Structure

Storing "cat", "car", "do"



What is the search complexity?

What is the search complexity?

Search Complexity:

- ❑ Search, insert, and delete for a key of length L takes $O(L)$ character steps—completely independent of how many millions of words are in the dataset!

L represents the length of the string/key being searched (the total number of characters, letters, or digits in the key).

- ❑ Space Complexity: $O(N * L * |\Sigma|)$ where N is the number of keys, L is max length, and $|\Sigma|$ is alphabet size (e.g., $|\Sigma| = 26$ for English, 256 for extended ASCII).

Node structure and implementation

Python

```
class TrieNode:
    def __init__(self):
        self.children = {} # or array of size 26
        self.is_end_of_word = False # boolean flag
```

children: A hash map, dictionary, or array storing links to child nodes.

is_end_of_word: A boolean set to **True** **only if** a complete word terminates at this specific node.

Why is `_end_of_word`: Prefixes vs. Complete Words

Imagine you insert two words into an empty Trie:

1. "app"
2. "apple"

Because "app" is a prefix of "apple", both words share the exact same path of character nodes in the Trie: (root) -> 'a' -> 'p' -> 'p' -> 'l' -> 'e'

Now, imagine someone searches for the word "app".

- The search algorithm follows the nodes 'a', 'p', 'p'.
- It successfully finds node 'p'!
- But how does the algorithm know if "app" was **actually inserted as a stand-alone word**, or if it just exists as a passive prefix along the path to "apple"?

Without `is_end_of_word`: If another user searches for "ap" or "appl", the Trie would incorrectly report that those are valid stored words simply because the path of nodes exists!

Real-World Applications of Tries

Search Engine Autocomplete:

- Suggests matching terms in real-time as a user types into Google or Amazon.

IDE Code Completion & IntelliSense:

- Fast dropdown listing of available methods when typing `object.get....`

Spellcheckers & Auto-Correct:

- Validates words character-by-character and finds nearest dictionary prefixes.

Network IP Routing (Longest Prefix Matching):

- Routers match destination IP address prefixes to forward internet packets efficiently.

```
object.get...  
|  
├─ getAttribute(name)  
├─ getBoundingClientRect()  
├─ getElementsByClassName(names)  
├─ getElementsByTagName(name)  
├─ getItem(key)  
└─ getHeader(name)
```

Choosing the Right Data Structure

Feature	Hash Table	Trie (Prefix Tree)	Binary Search Tree
Exact Key Lookup	$O(1)$ Expected	$O(L)$ (Key length)	$O(\log N)$
Prefix Matching	Not Supported / $O(N)$	$O(P)$ (Prefix length)	$O(\log N + K)$
Ordered Traversal	No	Yes (Alphabetical)	Yes (In-order)
Primary Use Case	Fast general lookups, caches, dicts	Autocomplete, dictionaries, IP routing	Sorted datasets, range queries

$$\text{Total Time} = \underbrace{O(\log N)}_{\text{Find first matching node}} + \underbrace{O(K)}_{\text{Traverse } K \text{ matching nodes}} = O(\log N + K)$$

Feature	Hash Table	Trie (Prefix Tree)	Binary Search Tree
Exact Key Lookup	O(1) Expected	O(L) (Key length)	O(log N)
Prefix Matching	Not Supported / O(N)	O(P) (Prefix length)	O(log N + K)
Ordered Traversal	No	Yes (Alphabetical)	Yes (In-order)
Primary Use Case	Fast general lookups, caches, dicts	Autocomplete, dictionaries, IP routing	Sorted datasets, range queries

K items that match a given prefix



Summary

- ❖ **Key Lookup** is the foundation of dictionary ADTs and data retrieval.
- ❖ **Direct Addressing** gives instant access but wastes memory; **Hashing** compresses large key spaces into manageable tables.
- ❖ **Collision Handling** (Chaining & Open Addressing) along with **Load Factor management** keeps hash tables running at peak **$O(1)$** speed.
- ❖ **Tries** trade slightly higher memory for unique capabilities: lightning-fast **prefix matching** and **autocomplete** applications.